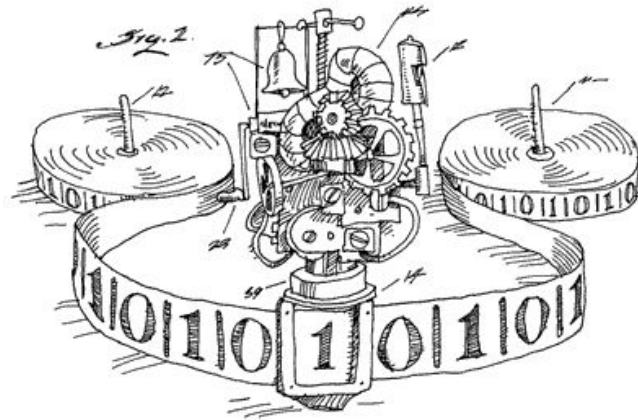
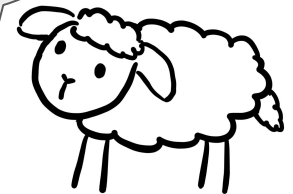


EECS 376: Foundations of Computer Science

Nicole Wein ("wine")
she/her



O(hi!)



"Of all the courses I've ever taken, this was one of them"
- New York Times Review of CS Courses

"We cannot recommend this course too highly"
- The Michigan Daily

Sheep art by Melissa Zhang: my sister-in-law
and mathematics professor at UC Davis

Course Staff

Instructors: 3



Me: Nicole Wein,



Emily Graetz,



Mahdi Cheraghchi

Admin Support: 1 Rose Sherry

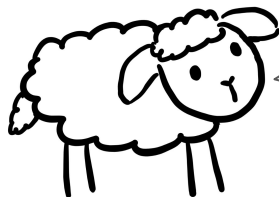
GSIs: 5

IAs: 4

Graders: ~10

Mascots: 1

You **can** attend:
- any lecture(s)
- any discussion(s)



Hi! My name is λ .
I'm a lamb, duh!

Learning Objectives

1. Given a challenging computational problem:
 - Determine whether or not a computer can solve it.
 - If *not*, prove it!
 - If *so*:
 - Design an algorithm for it
 - Derive its worst-case running time
 - Prove the correctness of the algorithm

Wait, some
problems are
unsolvable?



Learning Objectives

2. Get comfortable with major techniques and paradigms:

- Divide-and-conquer, greed, dynamic programming, the power of randomness
- Computability
- NP-completeness and approximation algorithms
- Cryptography

Problems that are **easy** for a computer

Problems that are **impossible** for a computer

Problems that are "**probably hard**" for a computer

Using "probably hard" problems for our benefit (hiding secrets)

- ## 3. Approach computational problems in a principled and rigorous way
- In this course you will be asked to think in ways you have never thought before.
 - This will make you a better computer scientist and problem solver.

Course Outline

- Algorithm Design & Analysis (7 lectures)
- Computability theory (6 lectures)
- Complexity theory (6 lectures)
- Randomized algorithms (3 lectures)
- Cryptography (3 lectures)
- Special topics (1 lecture)

Is this an EECS class?

- **Question:** Wolverine Access says it is an EECS class. Why does it feel like a math class?
- **Answer:** It's both!

The only way to answer the questions we raise is to **define mathematical models** and apply a **rigorous proof-based** methodology to the questions.

- **Question:** Why do we do proofs?

Answer: 1. To be confident in correctness. 2. To understand.

Why study CS foundations?

(aka theoretical computer science)



Top 7 Reasons

1. Theoretical ideas underlie a lot of software (e.g. hardware design, Internet routing, cryptography, many more).

"Everyone knows Moore's Law — a prediction made in 1965 by Intel co-founder Gordon Moore that the density of transistors in integrated circuits would continue to double every 1 to 2 years...in many areas, performance gains due to improvements in **algorithms** have vastly exceeded even the dramatic performance gains due to increased processor speed." -Report to the President and Congress: Designing a Digital Future (2010)

2. It's very easy to write code that is either incorrect or will run for millions of years.

"For every complex problem there is an answer that is clear, simple, and wrong." -H. L. Mencken

Why study CS foundations?

(aka theoretical computer science)



Top 7 Reasons

- 3. In your job you might be asked you to solve a problem that's provably impossible. Save yourself the trouble!
- 4. It will help you ace interviews.
- 5. Foundational reasoning applies to any computer: past, present, or future!

Why study CS foundations?

(aka theoretical computer science)

Top 7 Reasons

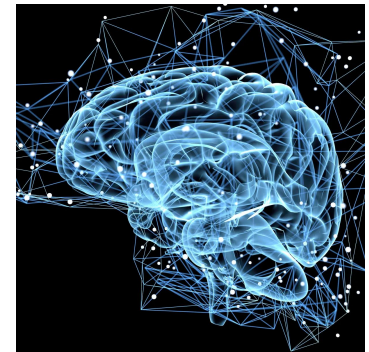
6. Computation is everywhere (not just in computers)



Economics / Game Theory
e.g. auctions, matching markets



Collective animal behavior



Your brain

7. Mathematical beauty

Struggling is good

This course asks you to comprehend deep ideas and solve challenging problems. You are supposed to struggle!

It is important to have wrong ideas. The discoverers of every idea in this course had many wrong ideas before having the right idea.

We are here to help! You will likely do better in this course if you ask questions during lecture, at office hours, and on Piazza.

An example of a good question to ask in class: "Can you repeat what you just said?"

The (AI)elephant in the room

What is the **point** of learning, designing, and analyzing algorithms anymore?

- AI is often **wrong** or **inadequate**: once you get to the frontier of the field, it cannot solve most problems
- You need **subject matter expertise** to **interact productively** with AI: ask the right questions and accurately interpret its output
- The real world is messy and you need to be able to **distill real-world problems** into well-defined computational problems

Administration

Website (eecs376.org) contains all resources:

- **Syllabus**
- **Schedule:** includes office hours
- **Course Notes:** Follows lectures quite closely (but not exactly)
- **Files:** HWs, (annotated) lecture slides, discussion worksheets, etc.
- **Lecture recordings**
- **Gradescope:** HW submission + HW/exam grades
- **Piazza:** questions (feel free to answer!)
- **Forms:** admin form (arising issues, prolonged illness), exam conflict form
- **My Proffice hours:**
 - Mondays 5-7pm in Beyster 3603
 - starting 2 weeks from today due to Labor Day
 - After class
 - By appointment

Administration

- 12 weekly HW assignments, due **Wednesdays 8pm**
 - Submissions open until 9:59pm, with 5% penalty
 - **No Late Submissions after 9:59pm**
 - Staff help only until 8pm
 - Two lowest scores will be dropped
 - Solutions published shortly after the deadline
 - HW 1 will be posted by Wednesday evening, and is due next Wednesday at 8pm
- **Midterm:** Monday, October 26, 7-9pm
- **Final:** Wednesday, December 16, 7-9pm
- If you have an exam conflict: eecs376.org -> *exam conflict form*

Collaboration

We encourage collaboration! See syllabus for more details

- When writing your solution, the **write-up must be done individually and entirely on your own**, and **you may not look at anyone else's write-up, including your collaborators'**.
- When turning in work that benefited from a collaboration, **you must acknowledge your collaborators** (as your answer to question 0 on the assignment).

Encouraged Collaboration	Unacceptable Collaboration
Brainstorming solution ideas, e.g., whether to use a reduction for a problem and ideas about what a suitable language for the reduction might look like	Walking through an important piece of the solution step-by-step, sharing solution sketches or drafts, or otherwise directly giving away your solution to someone, whether written or verbally
Helping others understand the problem statement and nuances of the problem	Providing your solution as a reference

(see syllabus)

⋮

AI use is not permitted on the HW

Grading

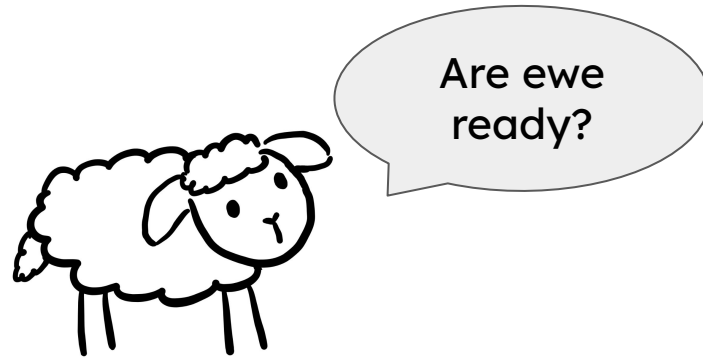
Lowest 2 HW scores dropped

Assignment	Weight
Homeworks	40%
Exams (midterm and final)	56-60%
Discussion attendance	0-3%
Course evaluations	0-1%
Total	100%

If you attend at least 10 out of the 14 discussions you get 3%.
Otherwise, your combined exam score will count as your score for this component.
Same goes for the course evaluations 1%.

Additionally, you need at least a 45% exam average to pass the course.

Now onto the course content...



Topic 1: Algorithms



+



Some historical events

Today + next time!

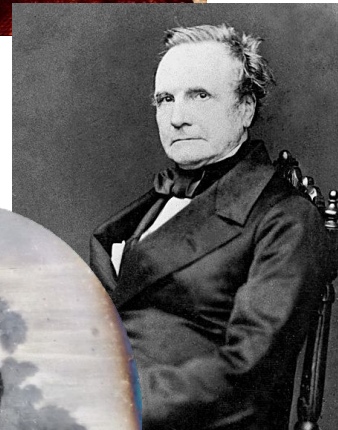
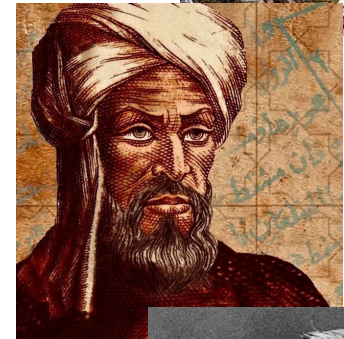
300BC: Euclid describes algorithms (for problems such as Greatest Common Divisor) that are still used today

825: Muhammad ibn Musa al-Khwarizmi writes “On the Calculation with Hindu Numerals.” The word “algorithm” will be named for al-Khwarizmi.

1837: Charles Babbage describes plans for Analytical Engine (mechanical general-purpose computer, never completed)

1842: In her notes on the Analytical Engine, Ada Lovelace writes the first algorithm specifically tailored for implementation on a computer (it calculated Bernoulli numbers)

(more history here: scottaaronson.blog/?p=524)



Review: Input Size

We express the running time of an algorithm as a function of its **input size**.

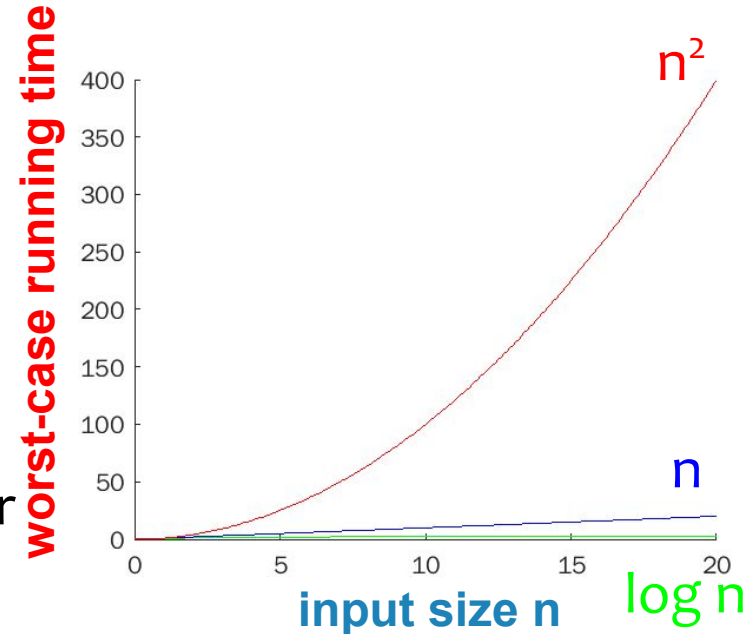
Common interpretations of input size:

- size of array = # elements (if the size of each element is negligible compared to the number of elements in the array)
- size of graph = # vertices + # edges
- size of integer = # **digits** (or #bits, doesn't matter which)
- **Rule of thumb:** size = # bits to represent input

Review: Running Time

We measure the efficiency of an algorithm by its **worst-case running time**: the maximum number of steps the algorithm can take on an **input of size n** .

Different computers running the same algorithm won't do exactly the same number of steps, but that's ok!



We abstract away such details with the help of **asymptotic notation**. We express running times in Big-O notation: e.g., $O(\log n)$, $O(n)$, $O(n^2)$ etc, where n is the **input size**.

"Efficient": running time polynomial in input size

Review: Asymptotic Notation

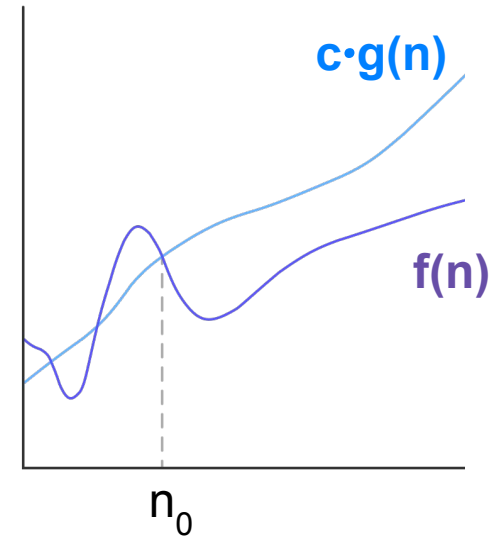
Formal definition of $f(n) = O(g(n))$:

there exist positive constants c, n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

$\geq$ for Ω instead of O

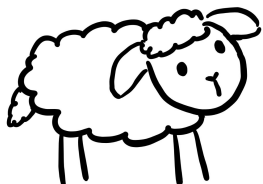
Informal definition of $f(n) = O(g(n))$: “at least” for Ω instead of O

$f(n)$ is asymptotically **at most** $g(n)$
(ignoring constant factors)



Which of these is NOT $O(n^2)$?

$3n^2$ n^2+n $\log n$ $n^4 - n^3$ $99999 \cdot n$

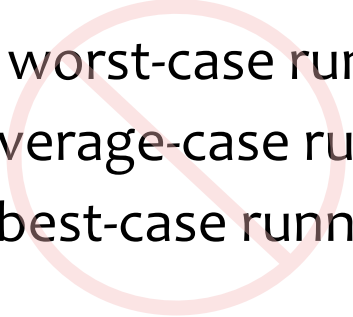


You will get more practice with this on the HW!

$f(n) = \Theta(g(n))$ means $f(n)$ is both $O(g(n))$ and $\Omega(g(n))$.

Watch out for this **Myth!**

“ O means worst-case running time
 Θ means average-case running time
 Ω means best-case running time”



Facts:

O , Θ , and Ω are mathematical definitions that aren't inherently associated with running time.

But in this course, we always talk about **worst-case** running time.

For more clarification on running times and asymptotics, check out *Handout 1: Running Times, Asymptotics* (eecs376.org -> files -> handouts)

Exponential vs. Polynomial



The λ -O-Matic performs 10^{11} operations/sec

	n=10	n=35	n=60	n=85
n^2	100 < 1 sec	1225 < 1 sec	3600 < 1 sec	7225 < 1 sec
n^3	1000 < 1 sec	43k < 1 sec	216k < 1 sec	614k < 1 sec
2^n	1024 < 1 sec	34×10^9 < 1 sec		

"Efficient": running time polynomial in input size

Exponential vs. Polynomial

An experiment to try with a child (or adult who hasn't taken 376):

If I pay you every day for a month, would you rather receive:

1. 1 penny on 1st day
2 pennies on 2nd day
4 pennies on 3rd day
8 pennies on 4th day
etc. (doubling each day)

OR

2. \$100 per day

"Efficient": running time polynomial in input size

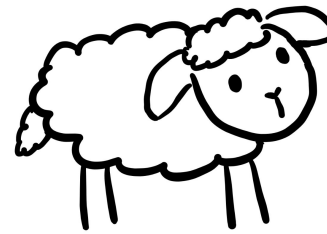
Now for the oldest algorithm...

The Tiling problem (aka gcd)

Input: n-digit integers $x \geq y \geq 0$, but not both =0.

Output: greatest common divisor $\text{gcd}(x,y)$
largest integer L that divides both x and y

In other words: largest integer tile size for exactly tiling both paths
(of length x and y)



I want to tile
two paths but
only buy one
size of tile

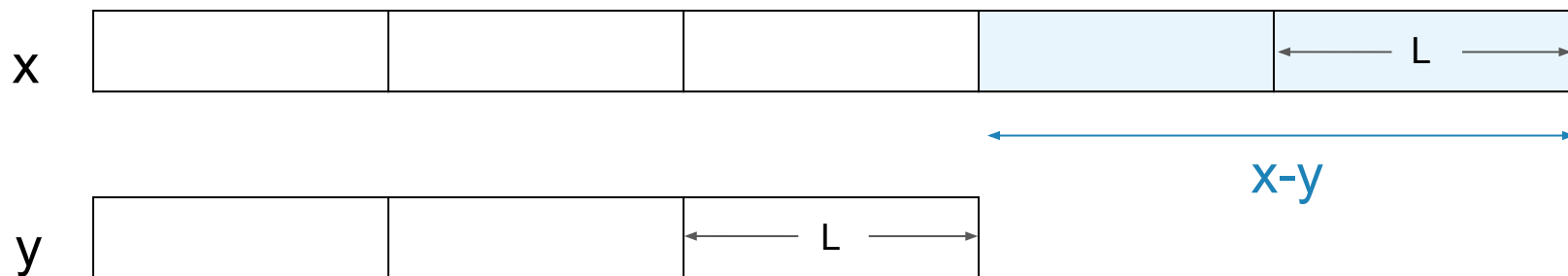
What is the simplest brute force algorithm you can think of?

Algorithm:

Is this an efficient algorithm? (in terms of the input size n)

Euclid's idea! (actually it was known before Euclid)

Claim: $\gcd(x, y) = \gcd(x-y, y)$.



Proof. We'll show that for any L : L divides x and $y \Leftrightarrow L$ divides $x-y$ and y .

1. ($\Leftarrow$) Suppose L divides both $x-y$ and y .

Why does this mean L divides x ?

2. ($\Rightarrow$) Suppose L divides both x and y .

Why does this mean L divides $x-y$?

Euclid's (more general) idea!

$$\gcd(x, y) = \gcd(x-y, y)$$

$$= \gcd(x-2y, y)$$

$$= \gcd(x-3y, y)$$

$$= \gcd(x-ky, y) \text{ for any } k \text{ such that } x-ky \geq 0$$

$$= \gcd(x \bmod y, y)$$

Now we have a smaller instance of the same problem.

What should we do? How do we turn this **identity** into a **program** for computing gcd?